ECE 276A Project 2: LiDAR-Based SLAM

Aditya Wirayutha Kaya Simto
Department of Computer Science and Systems Engineering
Kobe University

I. Introduction

Indoors there is no GPS, and odometry drifts: on dataset 20 it ends 2.77 m from the start after a 95.9 m drive. SLAM (Simultaneous Localization and Mapping) estimates the path and the map together, so that seeing the same wall twice corrects the path.

The system has four stages for a differential-drive robot with encoders, an IMU, a 2-D LiDAR and a Kinect [1] (Fig. 1): (1) encoder and IMU odometry, (2) ICP scan matching, (3) pose graph optimization with loop closures in GTSAM [3], and (4) occupancy grid and floor texture mapping with the optimized poses. The final path ends 0.12 m (dataset 20) and 0.22 m (dataset 21) from the start, compared to 2.7 to 3.3 m for odometry or ICP alone.

II. Problem Formulation

Pose. $\mathbf{x} = (x, y, \theta)$ with $\theta \in [-\pi, \pi)$, and matrix form

$$T(\mathbf{x}) = \begin{bmatrix} R(\theta) & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & x \\ \sin\theta & \cos\theta & y \\ 0 & 0 & 1 \end{bmatrix}, \quad (1)$$

which maps a body point to the world: $\mathbf{p}_W = R\mathbf{p}_B + \mathbf{t}$. The pose of j relative to i is $T_i^{-1}T_j$. For scan k, $T_k = T(\mathbf{x}_k)$ has rotation $R_k = R(\theta_k)$ and translation $\mathbf{t}_k$. The unknowns are the path $\{\mathbf{x}_0, \ldots, \mathbf{x}_{K-1}\}$ (one pose for each of the K LiDAR scans), the occupancy map m and the floor color map c.

Motion model. With speed v_t, yaw rate ω_t, time step τ_t:

$$\theta_{t+1} = \theta_t + \omega_t\tau_t, \quad \begin{bmatrix} x_{t+1} \\ y_{t+1} \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \end{bmatrix} + v_t\tau_t \begin{bmatrix} \cos\theta_{t+1} \\ \sin\theta_{t+1} \end{bmatrix}. \quad (2)$$

Registration. For point sets $\{\mathbf{p}_k\}$, $\{\mathbf{q}_k\}$ with known pairs,

$$(R^*, \mathbf{t}^*) = \arg\min_{R,\,\mathbf{t}} \sum_{k=1}^{N} \|\mathbf{q}_k - (R\mathbf{p}_k + \mathbf{t})\|^2 \quad \text{subject to} \quad R^\top R = I,\ \det R = 1. \quad (3)$$

ICP solves the same problem when the pairs are unknown.

Pose graph. The nodes are the poses $\mathbf{x}_0, \ldots, \mathbf{x}_{K-1}$, and two kinds of edges connect them. Odometry edges follow the motion model: the measured motion $\mathbf{u}_k = (\mathbf{u}_k^{xy}, u_k^\theta)$ from scan $k-1$ to scan k, in the frame of $k-1$, predicts the next pose,

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k, \quad f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \mathbf{t} + R(\theta)\,\mathbf{u}^{xy} \\ \theta + u^\theta \end{bmatrix}. \quad (4)$$

Loop-closure edges are observations: when scans i and j see the same walls, ICP between them measures $\mathbf{z}_{ij}$, the pose of j seen from i, with observation model

$$\mathbf{z}_{ij} = h(\mathbf{x}_i, \mathbf{x}_j) + \mathbf{v}_{ij}, \quad h(\mathbf{x}_i, \mathbf{x}_j) = \begin{bmatrix} R(\theta_i)^\top(\mathbf{t}_j - \mathbf{t}_i) \\ \theta_j - \theta_i \end{bmatrix}, \quad (5)$$

which is $T_i^{-1}T_j$ written as numbers. With Gaussian noise $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \Sigma_{\text{odo}})$, $\mathbf{v}_{ij} \sim \mathcal{N}(\mathbf{0}, \Sigma_{\text{loop}})$ and information matrices $\Omega = \Sigma^{-1}$, the most probable path minimizes the weighted squared residuals (angles wrapped to $[-\pi, \pi)$):

$$\mathbf{e}_k = \mathbf{x}_k - f(\mathbf{x}_{k-1}, \mathbf{u}_k), \qquad \mathbf{e}_{ij} = \mathbf{z}_{ij} - h(\mathbf{x}_i, \mathbf{x}_j),$$
$$\mathcal{J}(\mathbf{x}) = \tfrac{1}{2}\sum_k \mathbf{e}_k^\top \Omega_{\text{odo}} \mathbf{e}_k + \tfrac{1}{2}\sum_{(i,j)} \mathbf{e}_{ij}^\top \Omega_{\text{loop}} \mathbf{e}_{ij}. \quad (6)$$

This is the lecture's Graph SLAM with the landmark term $\mathbf{z} - h(\mathbf{x}, \mathbf{m})$ replaced by $\mathbf{z}_{ij} - h(\mathbf{x}_i, \mathbf{x}_j)$: the map is not in the state but is built afterwards from the optimized poses. GTSAM stores both edge types as relative-pose factors. Since $\sigma_x = \sigma_y$, the two residual forms give the same cost (in x, y, $\mathbf{x}_k - f = -R(\theta_{k-1})(\mathbf{u}_k - h)$), and GTSAM's own error agrees for small angle errors (2.09836 vs. 2.09835 at 0.01 rad).

Occupancy grid. Each cell m_i holds the log-odds $\lambda_i = \log \frac{p(m_i=1|z_{1:t})}{p(m_i=0|z_{1:t})}$, with $\lambda_i = 0$ meaning unknown (50 %).

III. Technical Approach

Table I gives the input and output of each function in the code.

A. Odometry

Finds a first path from the wheels and the gyro. Input: encoder counts and IMU yaw rate. Returns one pose (x, y, θ) per encoder reading (4956 for dataset 20). Errors add up step by step, so the path drifts.

Step 1. Distance per tick, wheel diameter $D = 0.254$ m, 360 ticks per turn: $s = \pi D/360 \approx 0.0022$ m.

Step 2. Side travel from the counts FR, FL, RR, RL since the last reading:

$$d_R = \tfrac{FR+RR}{2}\, s, \qquad d_L = \tfrac{FL+RL}{2}\, s. \quad (7)$$

TABLE I

Functions in main.py and icp.py. N = number of points, K = number of LiDAR scans (4962 for dataset 20).

Function	Input	Returns
odometry(d)	sensor data	(4956, 3) array of (x, y, θ), one per encoder reading
lidar_points(d, k)	scan number k	(N, 2) points of scan k in the body frame (meters)
kabsch(P, Q)	two (N, d) arrays, row k paired with row k	R, t, rmsd: best rotation, translation, fit error
icp(source, target, T_initial)	two clouds, no pairs, a starting guess	T, error: $(d{+}1) \times (d{+}1)$ transform source→target, mean pair distance
scan_matching(d, odom)	data, odometry poses	(K, 3) poses, one per LiDAR scan
match_scans(d, poses_T, i, j, …)	two scan numbers	T_relative, inlier_ratio: pose of j relative to i, share of points within 10 cm
pose_graph(d, icp_poses)	data, (K, 3) ICP poses	(K, 3) optimized poses
occupancy_map(d, poses)	data, (K, 3) poses	map_info, grid, snapshots: 1600×1600 log-odds grid + 4 copies over time
camera_to_body()	none	3×3 rotation, camera frame → body frame
texture_map(d, dataset, poses)	data, poses	map_info, texture, snapshots: 1600×1600×3 color image + 4 copies

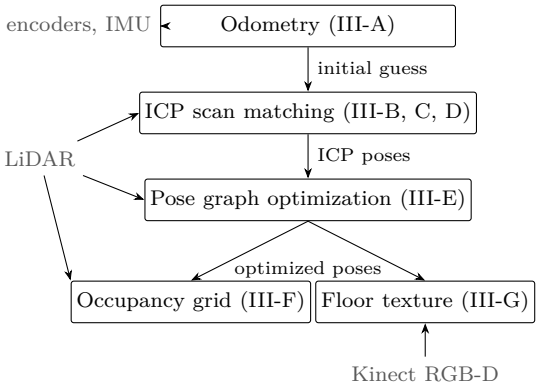


Fig. 1. Pipeline. Labels give the subsections of Section III.

Step 3. The center of a differential-drive robot moves by the average of the two sides, so with τ the encoder time step,

$$l = \tfrac{d_R + d_L}{2}, \qquad v = \tfrac{l}{\tau}. \tag{8}$$

Step 4. Yaw rate: the wheels give $\omega = (d_R - d_L)/(b\,\tau)$ with track width b, but wheels slip in turns, so ω is taken from the IMU z-axis gyro at the nearest IMU time stamp.
Step 5. Integrate with the Euler step (2), assuming v and ω constant over each τ.

Algorithm 1 Odometry

Input: counts FR, FL, RR, RL, stamps, IMU yaw rate
Returns: $\mathbf{x}_t = (x_t, y_t, \theta_t)$ for $t = 0, \ldots, N-1$

$\mathbf{x}_0 \leftarrow (0, 0, 0)$
for $t = 1, \ldots, N-1$ do
  $\tau \leftarrow \text{stamp}_t - \text{stamp}_{t-1}$
  $d_R \leftarrow (FR_t + RR_t)\,s/2$, $d_L \leftarrow (FL_t + RL_t)\,s/2$
  $v \leftarrow (d_R + d_L)/(2\tau)$, $\omega \leftarrow$ IMU yaw rate nearest in time
  $\theta_t \leftarrow \theta_{t-1} + \omega\tau$
  $(x_t, y_t) \leftarrow (x_{t-1}, y_{t-1}) + v\tau(\cos\theta_t, \sin\theta_t)$
end for

B. Kabsch algorithm (known pairs)

Finds the rotation and translation that best move point set P onto Q when row k of P belongs with row k of Q. Returns R, $\mathbf{t}$ and the RMSD (root mean square distance) of the aligned points. No iteration: one SVD gives the exact answer.
Step 1. Objective (3): $E(R, \mathbf{t}) = \sum_k \|\mathbf{q}_k - R\mathbf{p}_k - \mathbf{t}\|^2$.
Step 2. Optimal translation. Setting the gradient to zero,

$$\frac{\partial E}{\partial \mathbf{t}} = -2\sum_k (\mathbf{q}_k - R\mathbf{p}_k - \mathbf{t}) = 0 \;\Rightarrow\; \mathbf{t}^* = \bar{\mathbf{q}} - R\bar{\mathbf{p}}, \tag{9}$$

with centroids $\bar{\mathbf{p}} = \frac{1}{N}\sum_k \mathbf{p}_k$, $\bar{\mathbf{q}} = \frac{1}{N}\sum_k \mathbf{q}_k$. This is why centroids are used: the best translation always moves the source centroid onto the target centroid, so after centering both sets the translation drops out and the rotation can be solved alone.
Step 3. Centered points $\mathbf{p}'_k = \mathbf{p}_k - \bar{\mathbf{p}}$, $\mathbf{q}'_k = \mathbf{q}_k - \bar{\mathbf{q}}$. Substituting $\mathbf{t}^*$ and expanding, with $\|R\mathbf{p}'_k\| = \|\mathbf{p}'_k\|$:

$$E(R) = \sum_k \left(\|\mathbf{q}'_k\|^2 + \|\mathbf{p}'_k\|^2\right) - 2\sum_k \mathbf{q}'^{\top}_k R\,\mathbf{p}'_k. \tag{10}$$

The first sum does not depend on R, so minimizing E means maximizing the last sum.
Step 4. Trace form. Since $\mathbf{q}'^{\top} R\mathbf{p}' = \operatorname{tr}(R\,\mathbf{p}'\mathbf{q}'^{\top})$,

$$\begin{aligned} \textstyle\sum_k \mathbf{q}'^{\top}_k R\,\mathbf{p}'_k &= \operatorname{tr}(RH), \\ H &= \textstyle\sum_k \mathbf{p}'_k\mathbf{q}'^{\top}_k = P'^{\top}Q' \in \mathbb{R}^{d\times d}. \end{aligned} \tag{11}$$

Step 5. SVD. Every real matrix factors as

$$\begin{aligned} H = U\Sigma V^{\top}, \quad U^{\top}U = V^{\top}V = I, \\ \Sigma = \operatorname{diag}(\sigma_1 \geq \cdots \geq \sigma_d \geq 0), \end{aligned} \tag{12}$$

i.e. a rotation (or reflection), a scaling along the axes, and another rotation (or reflection). σ_i are the singular values.
Step 6. Bound. By the cyclic property of the trace,

$$\begin{aligned} \operatorname{tr}(RH) &= \operatorname{tr}(RU\Sigma V^{\top}) = \operatorname{tr}(\Sigma\, V^{\top}RU) \\ &= \textstyle\sum_i \sigma_i M_{ii}, \qquad M = V^{\top}RU. \end{aligned} \tag{13}$$

M is orthogonal, so $|M_{ii}| \le 1$ and $\operatorname{tr}(RH) \le \sum_i \sigma_i$. Equality holds for $M = I$, so $R^* = VU^\top$.

Step 7. Reflection fix. If $\det(VU^\top) = -1$, $VU^\top$ is a mirror, not a rotation. The best proper rotation flips the axis with the smallest σ_d:

$$R^* = V \operatorname{diag}\big(1, \dots, 1, \det(VU^\top)\big) U^\top, \tag{14}$$

which the code does by negating the last row of $V^\top$. A self-test rotates a random cloud by 40° and recovers R and $\mathbf{t}$ exactly.

Algorithm 2 Kabsch

Input: $P, Q \in \mathbb{R}^{N \times d}$, row k paired with row k
Returns: R, $\mathbf{t}$, RMSD
1: $\bar{\mathbf{p}}, \bar{\mathbf{q}} \leftarrow$ means of the rows; $P' \leftarrow P - \bar{\mathbf{p}}$, $Q' \leftarrow Q - \bar{\mathbf{q}}$
2: $H \leftarrow P'^\top Q'$ ▷ $d \times d$
3: $U, \Sigma, V^\top \leftarrow \mathrm{SVD}(H)$
4: $R \leftarrow V \operatorname{diag}(1, \dots, 1, \det(VU^\top))\, U^\top$
5: $\mathbf{t} \leftarrow \bar{\mathbf{q}} - R\bar{\mathbf{p}}$
6: return R, $\mathbf{t}$, $\sqrt{\frac{1}{N}\sum_k \|R\mathbf{p}'_k - \mathbf{q}'_k\|^2}$

C. ICP (unknown pairs)

Finds the transform T that moves a source point set onto a target point set when the pairs are unknown, by alternating two steps: guess the pairs (nearest neighbors), then solve Kabsch on them. Input: source, target, initial guess T^0. Returns the final T and the mean pair distance.

Step 1. Start from T^0 and build a KD-tree on the target once.

Step 2. Transform: $\tilde{\mathbf{p}}_k = R^n \mathbf{p}_k + \mathbf{t}^n$.

Step 3. Correspondence (nearest neighbor):

$$j(k) = \arg\min_j \|\mathbf{q}_j - \tilde{\mathbf{p}}_k\|.$$

Step 4. Outlier rejection:

$$\mathcal{I} = \{k : \|\mathbf{q}_{j(k)} - \tilde{\mathbf{p}}_k\| < d_{\max}\}.$$

Step 5. Update with Kabsch on the kept pairs, using the original $\mathbf{p}_k$, so the result is the full transform, not an increment:

$$(R^{n+1}, \mathbf{t}^{n+1}) = \mathrm{Kabsch}\big(\{\mathbf{p}_k, \mathbf{q}_{j(k)}\}_{k \in \mathcal{I}}\big).$$

Step 6. Error and stopping rule:

$$e^n = \frac{1}{|\mathcal{I}|} \sum_{k \in \mathcal{I}} \|\mathbf{q}_{j(k)} - \tilde{\mathbf{p}}_k\|,$$

$$\text{stop if } |e^{n-1} - e^n| < 10^{-6} \text{ or } n = 50.$$

Convergence. With $E(T, j)$ the squared error (3) for transform T and pairing j,

$$E(T^n, j^n) \;\ge\; E(T^n, j^{n+1}) \;\ge\; E(T^{n+1}, j^{n+1}), \tag{15}$$

because step 3 picks the closest pairs for the current T and step 5 picks the best T for those pairs. A non-increasing sequence bounded below by 0 converges, so ICP stops, but only at the nearest local minimum. A far-off T^0 gives wrong pairs from the first iteration, so a good initial guess is required. (With $d_{\max}$ the kept set changes, so the decrease is nearly, not strictly, monotone.)

Warm-up. The partial view is registered onto the full model and the result inverted: every view point has a partner on the model, but the hidden back of the model has none in the view. ICP is started from 12 yaws $(0°, 30°, \dots, 330°)$, each with the centroids aligned, using every 10th view point; the start with the lowest error is refined with all view points.

Algorithm 3 ICP

Input: source $\{\mathbf{p}_k\}$, target $\{\mathbf{q}_j\}$, T^0, $d_{\max}$
Returns: T (source → target), mean pair distance e
1: build a KD-tree of the target; $e^{-1} \leftarrow \infty$
2: for $n = 0, \dots, 49$ do
3: $\quad \tilde{\mathbf{p}}_k \leftarrow R^n \mathbf{p}_k + \mathbf{t}^n$ for all k
4: $\quad j(k) \leftarrow$ nearest target point to $\tilde{\mathbf{p}}_k$ ▷ new pairs
5: $\quad \mathcal{I} \leftarrow \{k : \|\mathbf{q}_{j(k)} - \tilde{\mathbf{p}}_k\| < d_{\max}\}$
6: $\quad e^n \leftarrow$ mean of $\|\mathbf{q}_{j(k)} - \tilde{\mathbf{p}}_k\|$ over $\mathcal{I}$
7: $\quad (R^{n+1}, \mathbf{t}^{n+1}) \leftarrow \mathrm{Kabsch}(\{\mathbf{p}_k\}, \{\mathbf{q}_{j(k)}\})_{k \in \mathcal{I}}$ ▷ new T
8: $\quad$ if $|e^{n-1} - e^n| < 10^{-6}$ then break
9: end for
10: return T^{n+1}, e^n

D. LiDAR scan matching

Finds the pose of every LiDAR scan by matching it with ICP against a keyframe scan, starting from the odometry motion. Returns K poses (4962 for dataset 20). Less drift than odometry, but still some.

Step 1. Ranges to body points: for beam n with range r_n, $\alpha_n = \alpha_{\min} + n\Delta\alpha$ and

$$\mathbf{p}_n = \big(r_n \cos\alpha_n + 0.133,\; r_n \sin\alpha_n\big), \quad r_{\min} < r_n < r_{\max}, \tag{16}$$

where 0.133 m (0.13323 in the code) is the LiDAR offset from the robot center [2]. $S_k = \{\mathbf{p}_n\}$ is the point set of scan k.

Step 2. Relative pose. A wall point has one world position: $T_i \mathbf{p}^{(i)} = T_j \mathbf{p}^{(j)} \Rightarrow \mathbf{p}^{(i)} = (T_i^{-1} T_j)\, \mathbf{p}^{(j)}$. So ICP of scan j onto scan i estimates $T_i^{-1} T_j$.

Step 3. Initial guess from odometry poses O, relative to keyframe kf:

$$T^0 = T_{\mathrm{kf}}^{-1}\, T_{k-1}\, (O_{k-1}^{-1} O_k). \tag{17}$$

$O_{k-1}^{-1} O_k$ is the odometry step in the robot frame, so it can be stacked on the ICP pose T_{k-1} even after the two paths drift apart.

Step 4. Match and compose: $T_{\mathrm{rel}} = \mathrm{ICP}(S_k, S_{\mathrm{kf}}, T^0)$ with $d_{\max} = 0.5$ m, then $T_k = T_{\mathrm{kf}} T_{\mathrm{rel}}$.

Step 5. Scan k becomes the keyframe when $\|\mathbf{t}_{\mathrm{rel}}\| > 0.5$ m or $|\theta_{\mathrm{rel}}| > 0.2$ rad. Consecutive scans are only 2 cm apart; ICP under-reports such a small motion in a hallway, and chaining 4962 such steps made the hallways too short.

Algorithm 4 Scan matching

Input: scans S_k, odometry poses O_k (nearest in time)
Returns: ICP poses $T_0, \dots, T_{K-1}$

$T_0 \leftarrow O_0$; kf $\leftarrow 0$
for $k = 1, \dots, K-1$ do
  $T^0 \leftarrow T_{\text{kf}}^{-1} T_{k-1} O_{k-1}^{-1} O_k$ ▷ initial guess
  $T_{\text{rel}} \leftarrow \text{ICP}(S_k, S_{\text{kf}}, T^0, d_{\max}{=}0.5)$
  $T_k \leftarrow T_{\text{kf}} T_{\text{rel}}$
  if $\|\mathbf{t}_{\text{rel}}\| > 0.5$ or $|\theta_{\text{rel}}| > 0.2$ then kf $\leftarrow k$
end for

E. Pose graph optimization

Finds the path that best agrees with all measurements at once. Input: ICP poses and scans. Returns K optimized poses. Iterative: each Levenberg-Marquardt step moves all poses a little.

Idea. Every pose is a node and every measurement between two poses is an edge, like a spring that says "pose j should sit here as seen from pose i". Odometry edges link consecutive poses; loop-closure edges link two visits to the same place. The optimizer moves all poses at once until the springs are as relaxed as possible; a smaller σ is a stiffer spring.

Step 1. Nodes: $\mathbf{x} = (\mathbf{x}_0, \dots, \mathbf{x}_{K-1})$, $3K = 14886$ unknowns, starting at the ICP poses. The map is not in the state; it is drawn afterwards from the final poses.

Step 2. Odometry edges. $\mathbf{u}_k = T_{k-1}^{-1} T_k$ is the ICP step from scan $k{-}1$ to scan k, in the frame of $k{-}1$. Composing the two matrices gives the motion model (4):

$$\begin{aligned} T_{k-1}U_k &= \begin{bmatrix} R_{k-1}R(u_k^\theta) & \mathbf{t}_{k-1} + R_{k-1}\mathbf{u}_k^{xy} \\ \mathbf{0}^\top & 1 \end{bmatrix} \\ \Rightarrow\ f(\mathbf{x}_{k-1}, \mathbf{u}_k) &= \begin{bmatrix} \mathbf{t}_{k-1} + R_{k-1}\mathbf{u}_k^{xy} \\ \theta_{k-1} + u_k^\theta \end{bmatrix}, \end{aligned} \tag{18}$$

with residual $\mathbf{e}_k = \mathbf{x}_k - f(\mathbf{x}_{k-1}, \mathbf{u}_k)$ and $\sigma = (0.01, 0.01, 0.005)$.

Step 3. Loop-closure edges. When the robot comes back to a place, ICP between the old scan i and the new scan j measures $\mathbf{z}_{ij}$, the pose of j seen from i, directly from the walls, so it does not contain the drift built up in between. The prediction is the relative pose (5):

$$\begin{aligned} T_i^{-1}T_j &= \begin{bmatrix} R_i^\top R_j & R_i^\top(\mathbf{t}_j - \mathbf{t}_i) \\ \mathbf{0}^\top & 1 \end{bmatrix} \\ \Rightarrow\ h(\mathbf{x}_i, \mathbf{x}_j) &= \begin{bmatrix} R_i^\top(\mathbf{t}_j - \mathbf{t}_i) \\ \theta_j - \theta_i \end{bmatrix}, \end{aligned} \tag{19}$$

with residual $\mathbf{e}_{ij} = \mathbf{z}_{ij} - h(\mathbf{x}_i, \mathbf{x}_j)$. Pairs are found in two ways: fixed-interval pairs $(i, i{+}10)$, with ICP $d_{\max} = 0.5\,\text{m}$; and proximity pairs among every 20th scan, closer than 2 m in the ICP path but at least 500 scans apart in time, with $d_{\max} = 1.0\,\text{m}$. ICP starts from $T_i^{-1}T_j$. ICP returns an answer even for scans that do not overlap, so a match is kept only if most of scan j lands on scan i:

$$\rho_{ij} = \frac{\#\{\mathbf{p} \in S_j :\ d(Z_{ij}\mathbf{p},\, S_i) < 0.1\,\text{m}\}}{|S_j|} \ \geq\ 0.6, \tag{20}$$

where Z_{ij} moves scan j by the ICP result and $d(\mathbf{q}, S_i)$ is the distance from $\mathbf{q}$ to the nearest point of S_i. Loop edges use $\sigma = (0.1, 0.1, 0.05)$, ten times looser than odometry.

Step 4. Prior: $\mathbf{e}_0 = \mathbf{x}_0 - \bar{\mathbf{x}}_0$ ($\bar{\mathbf{x}}_0$ the first ICP pose), $\sigma = 10^{-3}$. Step 10 explains why it is needed.

Step 5. Cost from probability. The estimate is the most probable path, $\arg\max_{\mathbf{x}} p(\mathbf{x} \mid \mathbf{u}, \mathbf{z})$. If each measurement depends only on the two poses it links and the noises are independent,

$$p(\mathbf{x} \mid \mathbf{u}, \mathbf{z}) \propto p(\mathbf{x}_0) \prod_k p(\mathbf{x}_k \mid \mathbf{x}_{k-1}, \mathbf{u}_k) \prod_{(i,j)} p(\mathbf{z}_{ij} \mid \mathbf{x}_i, \mathbf{x}_j). \tag{21}$$

Each factor is Gaussian, $\propto \exp(-\frac{1}{2}\mathbf{e}^\top \Omega\, \mathbf{e})$ with $\Omega = \Sigma^{-1} = \text{diag}(1/\sigma_x^2, 1/\sigma_y^2, 1/\sigma_\theta^2)$, so $-\log$ turns the product into the sum $\mathcal{J}$ of (6) plus $\frac{1}{2}\mathbf{e}_0^\top \Omega_0 \mathbf{e}_0$: the most probable path is the weighted least-squares path. Since the $\mathbf{u}_k$ come from the ICP poses, without loop closures the ICP path makes every residual zero. Only the loop closures change the path; the optimizer spreads their disagreement over all edges.

Step 6. Robust loop edges. Let $\varepsilon = \sqrt{\mathbf{e}^\top \Omega\, \mathbf{e}}$ be the size of a residual in sigmas. In least squares an edge costs $\frac{1}{2}\varepsilon^2$ and pulls with force ε (the slope), so a wrong loop closure 10σ off pulls with 10 and can bend the map. Loop edges therefore use the Huber cost, built from the pull: keep the least-squares pull for small errors, cap it at 1 for large ones, and integrate the pull to get the cost:

$$\begin{aligned} \psi(\varepsilon) &= \rho'_H(\varepsilon) = \min(\varepsilon, 1), \\ \varepsilon \leq 1:\ \ \rho_H(\varepsilon) &= \textstyle\int_0^\varepsilon s\, ds = \frac{1}{2}\varepsilon^2, \\ \varepsilon > 1:\ \ \rho_H(\varepsilon) &= \textstyle\int_0^1 s\, ds + \int_1^\varepsilon 1\, ds = \varepsilon - \frac{1}{2}. \end{aligned} \tag{22}$$

Both pieces give $\frac{1}{2}$ with slope 1 at $\varepsilon = 1$, so the cost has no jump or kink. As a probability, $\exp(-\rho_H)$ has heavier tails than a Gaussian: a large error is rare but possible, so a bad match is treated as a mistake and not explained by bending the map. Gauss-Newton needs squares, so each iteration writes $\rho_H \approx \frac{1}{2} w \varepsilon^2$ with $w = \psi(\varepsilon)/\varepsilon = \min(1, 1/\varepsilon)$, i.e. the loop edge's Ω is scaled by w ($w = 0.1$ for an edge 10σ off).

Step 7. Linearize. $\mathcal{J}$ is not quadratic in $\mathbf{x}$ because $R(\theta)$ contains cos and sin, so it is minimized by iterating: near the current guess $\mathbf{x}^{(n)}$, each residual is replaced by a straight line. A residual depends on two poses a, b:

$$\begin{aligned} \mathbf{e}(\mathbf{x}^{(n)} + \delta\mathbf{x}) &\approx \mathbf{e} + A\,\delta\mathbf{x}_a + B\,\delta\mathbf{x}_b, \\ A = \frac{\partial \mathbf{e}}{\partial \mathbf{x}_a}, &\qquad B = \frac{\partial \mathbf{e}}{\partial \mathbf{x}_b}, \end{aligned} \tag{23}$$

i.e. A says how the residual changes when pose a moves a little, and B the same for pose b. The derivative of a rotation is

$$\begin{aligned} \frac{dR}{d\theta} &= \begin{bmatrix} -\sin\theta & -\cos\theta \\ \cos\theta & -\sin\theta \end{bmatrix} = R\,S, \quad S = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \\ \frac{dR^\top}{d\theta} &= (RS)^\top = -SR^\top. \end{aligned} \tag{24}$$

Odometry: $\mathbf{e}_k^{xy} = \mathbf{t}_k - \mathbf{t}_{k-1} - R_{k-1}\mathbf{u}_k^{xy}$ and $e_k^\theta = \theta_k - \theta_{k-1} - u_k^\theta$, so entry by entry

$$A = \begin{bmatrix} -I_2 & -R_{k-1} S\, \mathbf{u}_k^{xy} \\ \mathbf{0}^\top & -1 \end{bmatrix}, \qquad B = I_3. \tag{25}$$

Loop: $\mathbf{e}_{ij}^{xy} = \mathbf{z}_{ij}^{xy} - R_i^\top(\mathbf{t}_j - \mathbf{t}_i)$ and $e_{ij}^\theta = z_{ij}^\theta - \theta_j + \theta_i$, so

$$A = \begin{bmatrix} R_i^\top & SR_i^\top(\mathbf{t}_j - \mathbf{t}_i) \\ \mathbf{0}^\top & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} -R_i^\top & \mathbf{0} \\ \mathbf{0}^\top & -1 \end{bmatrix}. \tag{26}$$

Step 8. Normal equations. With the straight-line residuals, each edge's cost becomes a quadratic in $\delta\mathbf{x}$:

$$\begin{aligned} &\tfrac{1}{2}(\mathbf{e} + J\delta\mathbf{x})^\top \Omega(\mathbf{e} + J\delta\mathbf{x}) \\ &= \tfrac{1}{2}\mathbf{e}^\top\Omega\mathbf{e} + \delta\mathbf{x}^\top J^\top\Omega\mathbf{e} + \tfrac{1}{2}\delta\mathbf{x}^\top J^\top\Omega J\,\delta\mathbf{x}, \end{aligned} \tag{27}$$

where J is the edge's row of blocks: A at pose a, B at pose b, zero elsewhere. Summing over edges, $\mathcal{J} \approx \text{const} + \delta\mathbf{x}^\top\mathbf{b} + \frac{1}{2}\delta\mathbf{x}^\top H\delta\mathbf{x}$ with $H = \sum J^\top\Omega J$ and $\mathbf{b} = \sum J^\top\Omega\,\mathbf{e}$. Setting the gradient $\mathbf{b} + H\delta\mathbf{x}$ to zero gives $H\,\delta\mathbf{x} = -\mathbf{b}$. H is a table of 3×3 blocks, one per pair of poses. An edge involves only poses a and b, so it adds to only four blocks of H and two of $\mathbf{b}$:

$$\begin{aligned} H_{aa} &\mathrel{+}= A^\top\Omega A, \quad H_{ab} \mathrel{+}= A^\top\Omega B, \\ H_{ba} &\mathrel{+}= B^\top\Omega A, \quad H_{bb} \mathrel{+}= B^\top\Omega B, \\ \mathbf{b}_a &\mathrel{+}= A^\top\Omega\,\mathbf{e}, \qquad \mathbf{b}_b \mathrel{+}= B^\top\Omega\,\mathbf{e}. \end{aligned} \tag{28}$$

The prior adds Ω_0 to H_{00} and $\Omega_0\mathbf{e}_0$ to $\mathbf{b}_0$.

Step 9. Update and iterate (Levenberg-Marquardt):

$$(H + \mu I)\,\delta\mathbf{x} = -\mathbf{b}, \qquad \mathbf{x}^{(n+1)} = \mathbf{x}^{(n)} + \delta\mathbf{x} \tag{29}$$

(angles wrapped). In every iteration $\mathbf{u}$, $\mathbf{z}$ and Ω stay fixed, while $\mathbf{e}$, A, B, H and $\mathbf{b}$ are recomputed at $\mathbf{x}^{(n)}$. With $\mu \to 0$ the step is Gauss-Newton, $-H^{-1}\mathbf{b}$; with large μ it is $-\mathbf{b}/\mu$, a short step downhill, since $\mathbf{b}$ is the gradient of $\mathcal{J}$. If $\mathcal{J}$ dropped, the step is kept and μ decreases; otherwise it is rejected and μ increases. GTSAM stops when $\mathcal{J}$ decreases by less than 10^{-5} (relative or absolute), or after 100 iterations.

Step 10. Prior and sparsity. Shifting every pose by the same $\mathbf{a}$, $\delta\mathbf{x}_a = \delta\mathbf{x}_b = (\mathbf{a}, 0)$, gives $A\delta\mathbf{x}_a + B\delta\mathbf{x}_b = -\mathbf{a} + \mathbf{a} = \mathbf{0}$ for odometry and $R_i^\top\mathbf{a} - R_i^\top\mathbf{a} = \mathbf{0}$ for loops, so $H\delta\mathbf{x} = \mathbf{0}$: relative measurements say nothing about where the whole map sits, and H is singular. The prior adds $\Omega_0 = 10^6 I$ to H_{00}, which removes this. Each edge touches 2 poses, so only $4962 + 2 \cdot 5578$ of the 4962^2 blocks of H are non-zero (about 0.07 %), and sparse elimination solves it fast.

Algorithm 5 Pose graph optimization

Input: ICP poses T_k, scans S_k
Returns: optimized poses $\mathbf{x}^*$

add prior on $\mathbf{x}_0$; add odometry edges $\mathbf{u}_k = T_{k-1}^{-1}T_k$
for each candidate pair (i, j) do
  $\mathbf{z}_{ij} \leftarrow \text{ICP}(S_j, S_i, T_i^{-1}T_j)$
  if $\rho_{ij} \geq 0.6$ then add loop edge $(i, j, \mathbf{z}_{ij})$, Huber
end for
$\mathbf{x}^{(0)} \leftarrow$ ICP poses
repeat ▷ $n = 0, 1, \ldots$
  $\mathbf{e}, A, B$ of every edge at $\mathbf{x}^{(n)}$; build H, $\mathbf{b}$
  solve $(H + \mu I)\,\delta\mathbf{x} = -\mathbf{b}$
  if $\mathcal{J}(\mathbf{x}^{(n)} + \delta\mathbf{x}) < \mathcal{J}(\mathbf{x}^{(n)})$ then
    $\mathbf{x}^{(n+1)} \leftarrow \mathbf{x}^{(n)} + \delta\mathbf{x}$; decrease μ
  else
    $\mathbf{x}^{(n+1)} \leftarrow \mathbf{x}^{(n)}$; increase μ
  end if
until $\mathcal{J}$ decreases by less than 10^{-5}, or 100 iterations
return $\mathbf{x}^{(n)}$

Example. Three poses on a line, all $\Omega = I$, the prior holding $x_0 = 0$. Odometry says $0 \to 1$ and $1 \to 2$ are each 1 m; a loop closure says $z_{02} = 1.8$ m. The residuals are $e_{01} = x_1 - x_0 - 1$, $e_{12} = x_2 - x_1 - 1$ and $e_{02} = 1.8 - (x_2 - x_0)$. Iteration 1, at $x_1 = 1$, $x_2 = 2$: residuals $(0, 0, -0.2)$, $\mathcal{J} = 0.02$. Their derivatives with respect to (x_1, x_2) are $(1, 0)$, $(-1, 1)$ and $(0, -1)$, so $H = \sum J^\top J = \left[\begin{smallmatrix} 2 & -1 \\ -1 & 2 \end{smallmatrix}\right]$, $\mathbf{b} = \sum J^\top e = (0, 0.2)$ and $\delta\mathbf{x} = -H^{-1}\mathbf{b} = (-0.067, -0.133)$. Iteration 2, at $x_1 = 0.933$, $x_2 = 1.867$: residuals $(-0.067, -0.067, -0.067)$, $\mathcal{J} = 0.0067$, $\mathbf{b} = 0$, so it stops. The 0.2 m disagreement is shared equally; with unequal Ω, the stiffer edges keep smaller residuals.

Checks: for one test factor, $\frac{1}{2}\sum(e/\sigma)^2 = 10.5001$ by hand and from factor.error(); steps 7 to 9 coded by hand on a 5-pose test loop reach GTSAM's optimum to 3×10^{-8}, and both Jacobians match finite differences. The small odometry σ is deliberate: with $\sigma = 5$ cm the loop closures shrank the path to 86.5 m (Table III). A likely reason: in a straight corridor, sliding one scan along the other barely changes the overlap, so loop matches are weak along the corridor, and loose odometry edges let them pull the hallways shorter.

F. Occupancy grid mapping

Finds which grid cells contain walls. Input: optimized poses and scans. Returns a 1600×1600 grid of log-odds (positive = wall, negative = free, 0 = never seen). Each beam lowers the cells it passes through and raises the cell where it ends.

Step 1. Beams in the world: $\mathbf{p}_W = R_k\mathbf{p} + \mathbf{t}_k$ and LiDAR origin $\mathbf{o} = T_k[0.133, 0, 1]^\top$.

Step 2. Cell index with resolution $r = 0.05$ m and map corner $\mathbf{m}_{\min} = (-30, -30)$ m: $\text{cell}(\mathbf{p}) = \lfloor(\mathbf{p} - \mathbf{m}_{\min})/r\rfloor$, on a 1600×1600 grid.

Step 3. Cells on each beam: $\{c_1, \ldots, c_L\} = \text{Bresenham}\big(\text{cell}(\mathbf{o}), \text{cell}(\mathbf{p}_W)\big)$.

Step 4. Bayes' rule for one cell: $p(m \mid z_{1:t}) = \eta\, p(z_t \mid m)\, p(m \mid z_{1:t-1})$, with normalizer $\eta = 1/p(z_t \mid z_{1:t-1})$.

Step 5. Divide the expression for $m = 1$ by the one for $m = 0$; η cancels:

$$\frac{p(m{=}1 \mid z_{1:t})}{p(m{=}0 \mid z_{1:t})} = g \cdot \frac{p(m{=}1 \mid z_{1:t-1})}{p(m{=}0 \mid z_{1:t-1})}, \quad g = \frac{p(z_t \mid m{=}1)}{p(z_t \mid m{=}0)}. \tag{30}$$

Step 6. Take the log, so the product becomes a sum:

$$\lambda_t = \lambda_{t-1} + \log g. \tag{31}$$

Step 7. Sensor model with 80 % trust ($g = 0.8/0.2 = 4$): the beam passed through $c_1, \ldots, c_{L-1}$, so $\lambda \mathrel{-}= \log 4$ (free), and it ended in c_L, so $\lambda \mathrel{+}= \log 4$ (occupied). Clip to $[-20, 20]$ so cells can still change. Every 2nd scan is used.

Step 8. Probability: $p(m{=}1) = 1 - \dfrac{1}{1 + e^\lambda}$ (the logistic function).

Algorithm 6 Occupancy grid

Input: poses T_k, scans S_k
Returns: log-odds grid λ

$\lambda \leftarrow 0$ (1600×1600)
for $k = 0, 2, 4, \ldots$ do ▷ every 2nd scan
  $\mathbf{o} \leftarrow \text{cell}(T_k[0.133, 0, 1]^\top)$
  for each $\mathbf{p} \in S_k$ do
    $c_1, \ldots, c_L \leftarrow \text{Bresenham}(\mathbf{o}, \text{cell}(R_k\mathbf{p} + \mathbf{t}_k))$
    $\lambda(c_1..c_{L-1}) \mathrel{-}= \log 4$; $\lambda(c_L) \mathrel{+}= \log 4$
  end for
  clip λ to $[-20, 20]$
end for

G. Floor texture mapping

Finds the floor color of each grid cell. The occupancy grid only says where walls are; the texture map adds what the floor looks like, and since it comes from a second sensor it also checks the poses by eye: a wrong pose paints the same floor patch in two places. Input: optimized poses, Kinect disparity and RGB images. Returns a $1600 \times 1600 \times 3$ color image on the same grid as the occupancy map. Each depth pixel becomes a 3-D point; floor points are moved to the world and painted with their color.

For every 5th Kinect image, with the nearest RGB image and pose in time:

Step 1. Depth from disparity d (handout calibration): $dd = -0.00304\,d + 3.31$, $z = 1.03/dd$.

Step 2. Matching RGB pixel for depth pixel (u, v) (column, row):

$$u_{\text{rgb}} = \frac{526.37\,u + 16662}{585.051}, \quad v_{\text{rgb}} = \frac{526.37\,v + 19276 - 7877.07\,dd}{585.051}. \tag{32}$$

Step 3. Pinhole model. Projection with intrinsics K:

$$z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad K = \begin{bmatrix} f & 0 & c_u \\ 0 & f & c_v \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ z \end{bmatrix} = z\,K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \tag{33}$$

so $x = (u - c_u)z/f$, $y = (v - c_v)z/f$, with $f = 585.05$, $c_u = 315.84$, $c_v = 242.94$ (optical frame: x right, y down, z forward).

Step 4. Optical to camera frame (x forward, y left, z up): $\mathbf{p}_C = (z, -x, -y)$.

Step 5. Camera to body: $\mathbf{p}_B = R_z(\psi)R_y(\phi)R_x(\gamma)\,\mathbf{p}_C + [0.18, 0.005, 0.36]^\top$, with roll $\gamma = 0$, pitch $\phi = 0.36$, yaw $\psi = 0.021$ rad. Each factor keeps its own axis fixed and rotates the other two, e.g.

$$R_y(\phi) = \begin{bmatrix} \cos\phi & 0 & \sin\phi \\ 0 & 1 & 0 \\ -\sin\phi & 0 & \cos\phi \end{bmatrix}. \tag{34}$$

The camera forward axis maps to $R[1, 0, 0]^\top = (0.936, 0.020, -0.352)$, i.e. 20.6° below horizontal.

Step 6. Floor filter: keep $0 < z < 5$ m and $|p_{B,z} - z_{\text{floor}}| < 0.05$ m, with $z_{\text{floor}} = -0.14$ m measured from the data (not 0), and the RGB pixel inside the image.

TABLE II
Trajectory quality.

Data	Method	Path	Gap to start	Cross-pass
20	odometry	95.9 m	2.77 m	18.7 cm
20	ICP	95.3 m	2.69 m	10.8 cm
20	pose graph	93.9 m	0.12 m	2.7 cm
21	odometry	91.8 m	3.27 m	17.1 cm
21	ICP	96.7 m	2.80 m	11.4 cm
21	pose graph	93.7 m	0.22 m	1.1 cm

TABLE III
Pose graph tuning, dataset 20.

Variant	Cross-pass	Path
ICP only, no pose graph	10.8 cm	95.3 m
fixed-interval closures only	11.4 cm	94.4 m
odometry $\sigma = 5$ cm, 0.02 rad	1.7 cm	86.5 m
odometry $\sigma = 1$ cm, 0.005 rad (used)	2.7 cm	93.9 m

Step 7. World and paint: $\mathbf{p}_W = R_k\,\mathbf{p}_{B,xy} + \mathbf{t}_k$, then $c\big(\text{cell}(\mathbf{p}_W)\big) \leftarrow \text{RGB}(u_{\text{rgb}}, v_{\text{rgb}})$.

Algorithm 7 Floor texture

Input: poses, disparity and RGB images
Returns: color map c ($1600 \times 1600 \times 3$)

$c \leftarrow$ black
for every 5th disparity image do ▷ all pixels at once
  RGB image and pose T_k nearest in time
  $dd, z \leftarrow$ depth (step 1); $u_{\text{rgb}}, v_{\text{rgb}}$ (step 2)
  $\mathbf{p}_C \leftarrow (z, -(u - c_u)z/f, -(v - c_v)z/f)$ ▷ steps 3, 4
  $\mathbf{p}_B \leftarrow R_z(\psi)R_y(\phi)R_x(\gamma)\mathbf{p}_C + [0.18, 0.005, 0.36]^\top$
  keep floor pixels (step 6)
  $c(\text{cell}(R_k\mathbf{p}_{B,xy} + \mathbf{t}_k)) \leftarrow \text{RGB}(u_{\text{rgb}}, v_{\text{rgb}})$
end for

IV. Results

ICP warm-up. The mean distance from each view point to the model is 1.2 to 3.2 mm for all eight views (Fig. 3).

Trajectories. Without ground truth, two checks are used: the robot returns to its start, so the gap to the start should be near 0; and cross-pass consistency, the median distance from a scan's points to points seen on a different pass (≥ 500 scans apart). A correct path makes both passes see the same wall in the same place.

ICP lowers the cross-pass error from 18.7 to 10.8 cm (dataset 20) and from 17.1 to 11.4 cm (dataset 21), but still ends 2.7 to 2.8 m from the start (Table II, Fig. 4). The pose graph closes the loop: 0.12 and 0.22 m, with 2.7 and 1.1 cm cross-pass error. It accepted 496 fixed-interval and 121 of 875 proximity closures on dataset 20, and 475 and 233 of 832 on dataset 21. On dataset 20, 758 of the 875 candidates pair passes in opposite directions, and only 11 % of those pass the inlier test, against 33 % of same-direction pairs; facing the other way, the 270° LiDAR likely sees different walls.

Fixed-interval closures alone change almost nothing, since scans 10 apart repeat what the odometry factors already state; the proximity closures remove the drift

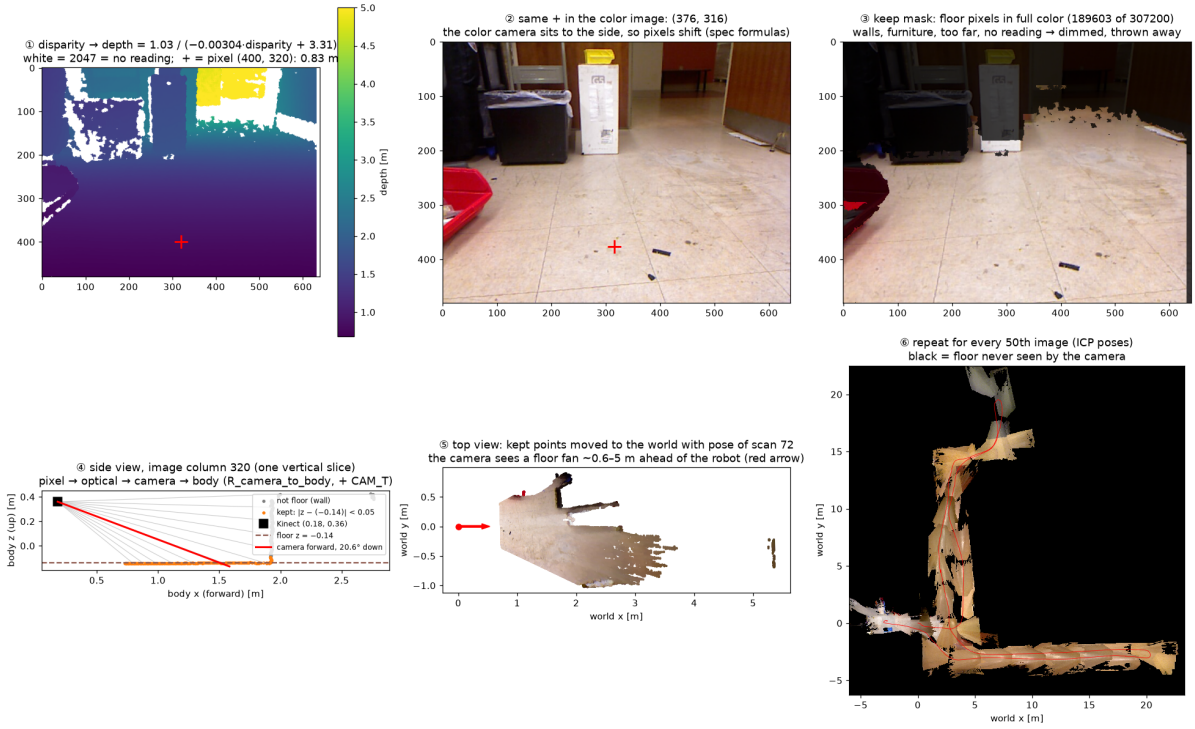


Fig. 2. One Kinect image through the texture pipeline (dataset 20, image 1): (1) depth from disparity; (2) the same pixel in the color image; (3) kept floor pixels; (4) side view of one column: rays from the tilted camera hit the floor at $z = -0.14$ m; (5) kept points in the world; (6) every 50th image painted with ICP poses.

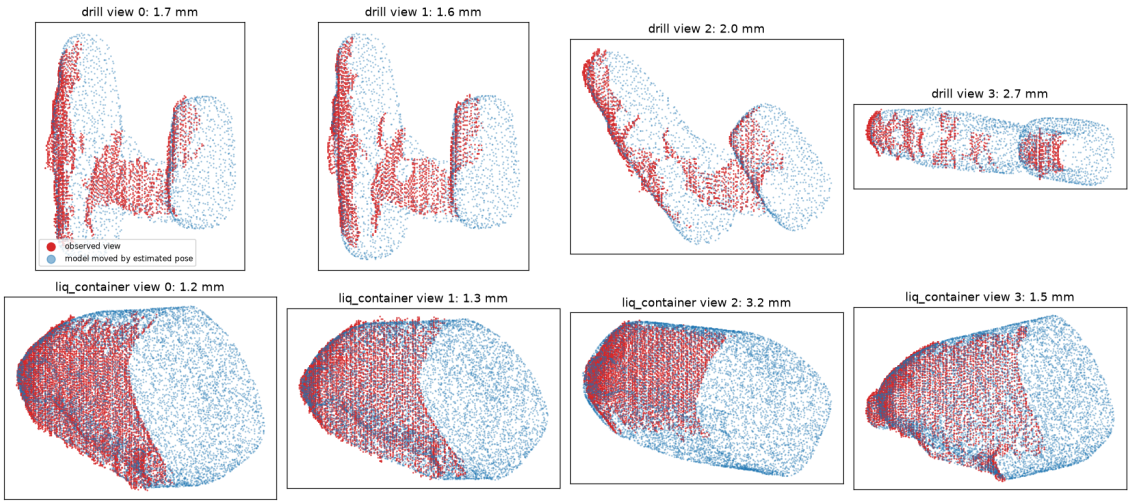


Fig. 3. ICP warm-up, top view. Red: partial view. Blue: model moved by the estimated pose. Title: mean point distance.

(Table III). Loose odometry noise gives the best cross-pass value but shortens the path by 9 m, so path length is checked as well.

Maps. Figs. 5 and 6 show the maps over time and ICP vs. optimized poses at the end. With ICP poses the return pass is about 11 cm off and the path end misses the start, which blurs the start area and the walls. With optimized poses both passes overlap, and the path forms an L about 23 m by 22 m. Run time: scan matching 43 s, pose graph 30 s, occupancy map 70 s, texture map 20 s.

Discussion. Odometry drifts from wheel slip and gyro bias; ICP uses the walls but only compares nearby scans, so it still drifts; the pose graph links scans far apart in time, which removes the drift. Three early failures and their fixes: matching to the previous scan shrank the hallways (keyframes), loose odometry noise let loop closures shrink the path (small σ), and ICP from a bad start locked onto a wrong pose (12 initial yaws).

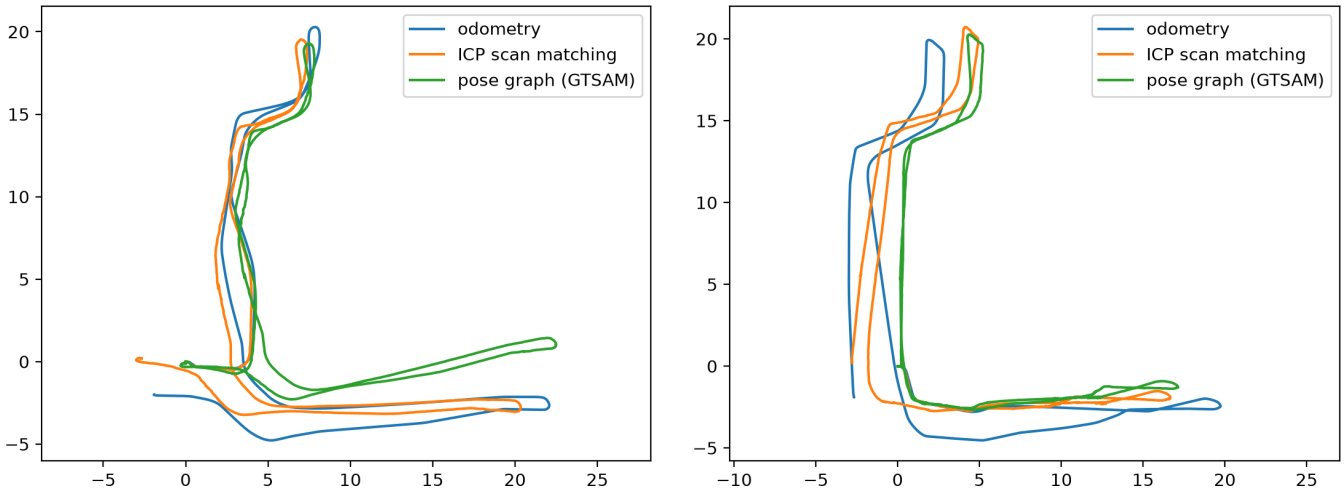


Fig. 4. Trajectories in meters: dataset 20 (left) and 21 (right).

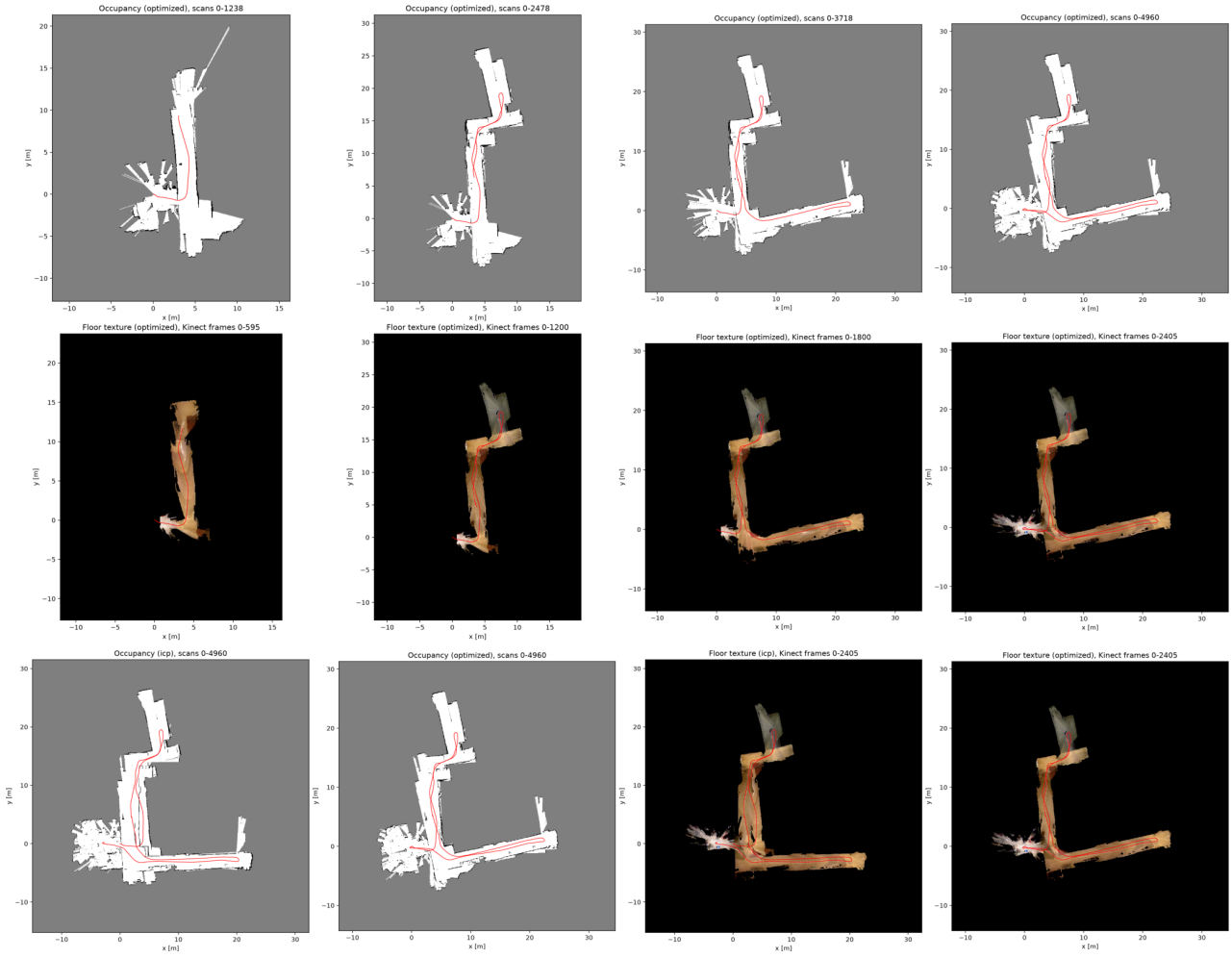


Fig. 5. Dataset 20. Top: trajectory and occupancy map over time (optimized poses; black = wall, white = free, gray = unknown, red = path so far). Middle: floor texture over time. Bottom: final maps, ICP vs. optimized poses.

Cross-pass consistency only checks that repeated passes agree, not the overall shape: on dataset 20 the two hallways meet at 91° with ICP poses and at 82° with optimized poses (directions fitted to the path). If the real hallways are perpendicular, closing the loop bent the corner by about 9°; ground truth would be needed

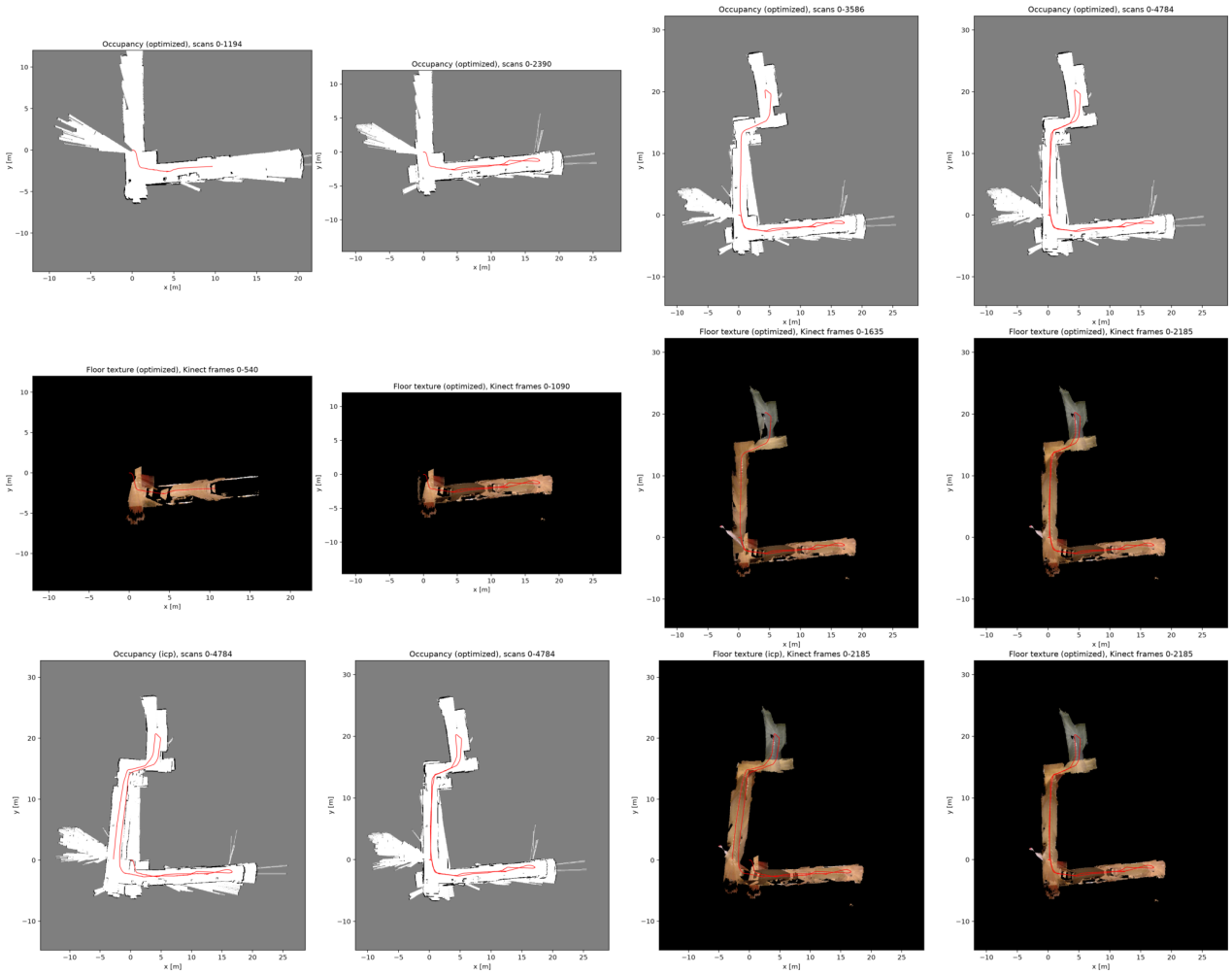


Fig. 6. Dataset 21, same layout as Fig. 5.

to confirm this.

Acknowledgment

Claude (an AI assistant) was used as a tutor for hints and code checks during the project, and to help draft this report from the results. Some of the code was run on Cinaps laptops.

References

[1] N. Atanasov, “ECE 276A: Sensing & Estimation in Robotics,” UC San Diego. [Online]. Available: https://natanaso.github.io/ece276a/

[2] “Robot configuration,” ECE 276A Project 2 handout (RobotConfiguration.pdf).

[3] F. Dellaert and GTSAM contributors, “GTSAM.” [Online]. Available: https://github.com/borglab/gtsam